HAND THIS WHOLE DOCUMENT TO CLAUDE CODE

Prospector — build spec

A small Go service that finds Kenyan companies likely to need backend/payments work, enriches them with real evidence, drafts a first-contact message per company for human review, and sends approved ones one at a time from your own Gmail. Postgres + Redis, one binary, server-rendered UI, no login.

> **NON-GOALS — REJECT THESE IF TEMPTED**
>
> No login or multi-user. No mass/bulk send — one message per click, throttled. No LinkedIn scraping or LinkedIn automation (account bans; do LinkedIn by hand). No CRM features, no pipeline kanban, no analytics dashboard, no Docker orchestration, no tests beyond the two acceptance checks in §10. No React — `html/template` + a little vanilla JS. Every hour spent past the box in §9 is an hour not spent messaging people.

---

1 · Stack and layout

```
Go 1.22+ · Postgres · Redis · net/http + chi · pgx · html/template · goquery
gomail (or net/smtp) · Anthropic API over plain HTTP (no SDK needed)

prospector/
  cmd/prospector/main.go        one binary: serve + background workers
  internal/store/               pgx queries, migrations/0001_init.sql
  internal/seed/                seed-list adapters (one file per source)
  internal/enrich/              polite crawler + extractors
  internal/score/               segment + score rules
  internal/draft/               Claude call, template fill
  internal/send/                SMTP send, throttle, log
  internal/web/                 handlers + templates/*.html
  internal/queue/               redis-backed job queue (LIST + BRPOPLPUSH)
  .env.example  Makefile  README.md

Run: `make db` (migrate) → `go run ./cmd/prospector` → http://localhost:8080
Workers run in-process as goroutines. No cron, no separate deployment.
```

2 · Environment

```
DATABASE_URL=postgres://user:pass@localhost:5432/prospector?sslmode=disable
REDIS_URL=redis://localhost:6379/0
HTTP_ADDR=:8080

# sending — Gmail with an APP PASSWORD, not your account password
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=anthonyalando8@gmail.com
SMTP_PASS=xxxx-xxxx-xxxx-xxxx
FROM_NAME=Anthony Alando
FROM_EMAIL=anthonyalando8@gmail.com
REPLY_TO=anthonyalando8@gmail.com

# drafting
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=claude-sonnet-4-5

# guardrails (read at runtime, not compiled)
DAILY_SEND_LIMIT=20
MIN_SEND_INTERVAL_SECONDS=90
CRAWL_DELAY_MS=1200
CRAWL_USER_AGENT=prospector/1.0 (+mailto:anthonyalando8@gmail.com)
```

3 · Postgres schema

```
create table company (
  id          bigserial primary key,
  name        text not null,
  domain      text unique,      -- normalised host, the dedupe key
  source      text not null,    -- clutch|google|jobboard|playstore|manual
  source_url  text,
  segment     text,             -- agency|incubator|hiring|sme
  score       int  not null default 0,
  status      text not null default 'new',
      -- new|enriched|drafted|approved|sent|replied
      -- |call|quoted|won|lost|skip
  location    text,
  notes       text,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);
create table contact (
  id bigserial primary key,
  company_id bigint not null references company(id) on delete cascade,
  kind text not null,        -- email|phone|whatsapp|form_url
  value text not null,
  person_name text, role text,
```

```
  quality int not null default 0,   -- named 3 > role 2 > info@ 1
  unique (company_id, kind, value)
);
create table signal (        -- the evidence a draft may cite
  id bigserial primary key,
  company_id bigint not null references company(id) on delete cascade,
  kind text not null,
      -- hiring_backend | no_checkout | payment_complaint
      -- | tech:flutter | tech:woocommerce | paybill_listed
      -- | portfolio_frontend_only | stale_site | app_reviews
  detail text not null,      -- VERBATIM quote or fact, max ~300 chars
  url text,
  created_at timestamptz not null default now()
);
create table page_cache (
  url text primary key,
  status int,
  body text,
  fetched_at timestamptz not null default now()
);
create table draft (
  id bigserial primary key,
  company_id bigint not null references company(id) on delete cascade,
  contact_id bigint references contact(id),
  channel text not null,     -- email|whatsapp|phone_script
  subject text, body text not null,
  evidence_used text,        -- which signal ids the model cited
  state text not null default 'pending',
      -- pending|approved|sent|discarded|needs_manual
  edited_by_human bool not null default false,
  created_at timestamptz not null default now()
);
create table send_log (
  id bigserial primary key,
  draft_id bigint not null references draft(id),
  company_id bigint not null references company(id),
  to_value text not null, channel text not null,
  sent_at timestamptz not null default now(),
  provider_response text,
  followup_1_due date, followup_2_due date, replied_at timestamptz
);
create index on company (status, score desc);
create index on signal (company_id);
create index on send_log (sent_at);
```

4 · What Redis is for

Key	Use
`queue:enrich` · `queue:draft`	Job lists of company ids. Workers use `BRPOPLPUSH` into a processing list so a crash doesn't lose the job.
`crawl:host:{host}`	Per-host rate limit — `SET NX EX 2` before each fetch; if it exists, requeue with delay. This is the politeness guarantee.
`send:day:{YYYY-MM-DD}`	Counter, `INCR` on each send, refuses past `DAILY_SEND_LIMIT`. Protects the Gmail account.
`send:last_at`	Timestamp enforcing `MIN_SEND_INTERVAL_SECONDS` so sends look human, never a burst.
`seen:domain:{domain}`	Fast duplicate check during seeding, before touching Postgres.

5 · Pipeline

```
SEED → company rows (status=new). Adapters, each independent, each behind one
       button in the UI plus a CSV import for hand-found names:
  jobboard   brightermonday.co.ke · fuzu.com · myjobmag.co.ke — search "backend",
             "golang", "django", "node", "software developer"; capture employer name,
             ad URL, and the ad text (it becomes a hiring_backend signal, verbatim)
  clutch     clutch.co + goodfirms.co listings filtered to Kenya
  google     plain SERP fetch for a list of queries in config; take result titles+URLs
  playstore  Kenyan apps in Finance/Shopping/Transport: developer site + low-star
             reviews mentioning payment/M-Pesa failures
  manual     paste CSV: name, domain, notes  ← use this tonight, before the crawler
  Normalise domain (strip www, lowercase). Skip if seen. Skip obvious non-prospects
  (uber.com, safaricom.co.ke, recruiters, university career pages).

ENRICH → contacts + signals (status=enriched). Per company:
  robots.txt check (skip disallowed paths) · fetch home, /about, /contact, /careers,
  /pricing — max 8 pages, 1 req/1.2 s per host, cache every page in page_cache
  extract  emails via regex, drop role@ duplicates, rank named > role > info@
           phones: /(\+?254|0)[17]\d{8}/ normalised to +254…
           tech signals from HTML: woocommerce · shopify · wp · firebase · flutter
           checkout presence: "add to cart" | "checkout" | "pay now" | "lipa"
           paybill/till numbers mentioned in text
           freshness: latest visible year / copyright, blog dates → stale_site
  every signal stores a VERBATIM detail + url. No paraphrase at this stage.

SCORE → segment + score (rules, not a model):
  segment  jobboard source → hiring · clutch/goodfirms or "we build" copy → agency
           incubator lists (manual CSV) → incubator · else → sme
  score    +3 hiring_backend · +3 payment_complaint · +2 no_checkout but sells ·
           +2 named contact · +2 agency portfolio has no backend/API mention ·
           +1 phone found · −2 stale_site · −3 no contact of any kind
  Sort desc. Only the top rows get drafted — drafting costs API money.

DRAFT → one row in draft (status=drafted). See §7 for the exact prompt contract.
  If the model returns no citable signal → state=needs_manual, do not fabricate.

REVIEW & SEND → human only (§6).
```

6 · UI and routes

Route	Screen
`GET /`	Today: sends used vs limit, follow-ups due today, counts by status, four seed buttons, CSV import box.
`GET /companies?segment=&status=`	Table sorted by score: name, segment, score, best contact, signal count, status. Row click → detail.
`GET /companies/{id}`	Contacts, all signals with source links, notes field, buttons: enrich · draft · skip.
`GET /review` ← the screen you live in	One pending draft at a time: company facts and signals on the left, editable subject/body textarea on the right. Buttons: **Save & send** · Save only · Skip · Needs manual. Editing sets `edited_by_human`. Keyboard: ⌘↵ send, → skip.
`POST /drafts/{id}/send`	Checks daily limit + interval, sends via SMTP, writes send_log with follow-up dates (+3 d, +8 d), sets company status=sent. Refuses if the body still contains a `[BRACKET]` placeholder — that guard alone will save you from a humiliating send.
`GET /followups`	Due today, with the original message shown and the two follow-up templates pre-filled.
`POST /companies/{id}/status`	Mark replied / call / quoted / won / lost. Whatsapp and phone channels are marked sent manually — the tool never dials or messages WhatsApp.

Plain HTML, one stylesheet, no build step. Every list is server-rendered; the only JS is the keyboard shortcuts and a character counter on the DM template.

7 · The drafting contract

The model writes **only two things**: the opening observation and a one-line problem hypothesis. Everything else is your template text, filled in code. This is what keeps 100 messages sounding like you rather than like a bot, and makes fabrication structurally impossible.

```
SYSTEM
You write one-line openers for a Nairobi backend engineer's cold outreach. You may
only state facts present in the SIGNALS provided. If the signals do not support a
specific, concrete observation, reply exactly: INSUFFICIENT.
Never invent clients, revenue, team size, or technology. No flattery, no adjectives
like "impressive" or "exciting". Kenyan business English, plain and direct.

USER
Company: {name} · {domain} · segment {segment}
Signals:
- {kind}: "{detail}"  ({url})
- ...
Return JSON only:
{"observation": "<=25 words, one concrete thing you noticed, no compliment",
 "hypothesis":  "<=20 words, the likely problem this implies for them",
 "signal_ids":  [ids you used]}

CODE THEN FILLS the audience template from the Outreach Kit:
  observation → first line · hypothesis → the sentence before the ask
  the ask, price, proof links, signature → always literal template text
REJECT the draft (state=needs_manual) if: response is INSUFFICIENT, signal_ids is
empty, observation contains a number not present in the signals, or the observation
is longer than 25 words. Log the reason.
COST guard: draft only the top-scoring N (default 40) and cache by company_id —
never re-draft on page refresh.
```

---

8 · Sending rules — do not let Claude relax these

One send per click. No “send all approved”. The button sends exactly one message, then the queue advances.

Your real Gmail with an app password, plain text body, no tracking pixel, no link shortener, no HTML template, one link maximum. Those are the three things that put cold mail in spam.

Ramp: 10 sends on day one, 15 on day two, then 20 — `DAILY_SEND_LIMIT` caps it, and 90 s minimum between sends.

Never contact the same domain twice within 30 days: enforce in `POST /send` with a query against send_log, not just in the UI.

Suppression list table + honour any “stop” reply immediately; add a one-line “tell me to stop and I won't write again” to the email footer. It costs nothing and it is the difference between outreach and spam.

9 · Build order, time-boxed

Box	Chunk to hand Claude, in this order
45 min	**1 · Skeleton + CSV import + review/send.** Schema, main.go, `/companies`, `/review` with a hand-written body, SMTP send with limits. *At the end of this box the tool is already useful — you can work a hand-built list tonight.*
60 min	**2 · Enrich worker.** Redis queue, polite fetcher with cache, email/phone/tech/checkout extractors, signals in the detail page.
45 min	**3 · One seed adapter** — job boards only, because that segment converts best. Others later, and only if the list runs dry.
45 min	**4 · Scoring + draft worker** with the §7 contract and the rejection rules.
30 min	**5 · Follow-ups screen** and status buttons. Stop here. Everything else is a want, not a need.

> HARD GATE
>
> After box 1, stop coding and send ten messages by hand from the tool. If box 2 isn't running by the end of the evening, ship what you have and spend tomorrow messaging, not debugging a crawler. The tool exists to make 100 conversations cheaper — it earns nothing itself.

10 · Acceptance checks

1. Import a 20-row CSV, enrich, draft, and review: every draft either cites a signal visible on the company page, or is marked `needs_manual`. No draft states a fact you cannot find in the signals list.

2. Send to your own second address: arrives in the inbox (not spam), plain text, correct reply-to, no placeholder brackets, send_log row written with both follow-up dates.

3. Try to exceed the daily limit and to send to the same domain twice: both refused with a clear message on screen.